I would like to work on, one is experimenting with creating my own (small) LLM or (not sure that is the right terminology) smart tool and training it on the lore from the fantasy/sci-fi world I have been building. I think the MacOS (M2) has enough bandwidth to handle it, at least on a small scale. Not sure about the Linux laptop.

I have explored previously a little bit some of the relevant topics. I am pretty good with python and SQL, can manage C/C++ if I need to, and am interested in learning Rust, and I have a fundamental understanding of big data, linear algebra, statistics and so on, though I am certainly not an expert data scientist in the modern sense and need help with terminology, newer technologies. This would be a hobby project, not something with any commercial or even public/open intensions, at least not initially.

Yes. And the biggest suggestion is this:

Do not start by "creating your own LLM" from scratch. Start by adapting a small existing model to your world.

For a hobby project on an M2 Mac, that is the sane path. Apple's MLX stack is built for Apple silicon, and MLX-LM explicitly supports running and fine-tuning LLMs on Apple silicon, including low-rank fine-tuning and quantized models. Hugging Face's PEFT docs likewise point to LoRA as the standard low-cost way to adapt a base model without retraining all parameters. (GitHub)

What I would do is a staged path:

Stage 1: Build a lore corpus before touching training.
Get your world material into clean text files with stable metadata: source, character, place, era, faction, reliability, and canon level. Split it into chunks that are coherent on their own. This matters more than people think. If your corpus is muddy, your model will be muddy.

Stage 2: Make a retrieval system first.
Before fine-tuning anything, build a small RAG prototype: embeddings + vector search + a local model that answers using retrieved lore chunks. This gives you fast feedback on whether the problem is really "the model lacks knowledge" or just "the context isn't being supplied well." In many lore projects, a good retrieval layer gets you most of the value with far less work and risk.

Stage 3: Then do a small LoRA fine-tune.
Take a small instruct model and fine-tune it on your lore voice, terminology, naming conventions, and house style. MLX-LM supports fine-tuning on Apple silicon, and PEFT/LoRA is the obvious method for a machine in your class because it keeps memory and storage demands down. (GitHub)

A good first target is not "make it know everything." It is narrower:

- answer consistently about your setting
- use your terminology correctly
- keep tone and style aligned with your lore
- avoid hallucinating against canon

That is much more achievable.

A practical first stack on the M2 Mac would be:

- **Python** for data prep and experiments
- **MLX / MLX-LM** for Apple-silicon-native local work (GitHub)
- **PEFT / LoRA** concepts for the adapter training approach (Hugging Face)
- **llama.cpp or Ollama** for easy local inference and testing of finished models/adapters (GitHub)

The Linux laptop may still be useful, but I would treat it as the secondary box for preprocessing, corpus management, evaluation scripts, SQL work, and maybe inference. The M2 Mac is the better first home for local fine-tuning because Apple's MLX ecosystem is specifically aimed at Apple silicon. (GitHub)

A very reasonable sequence of small steps would be:

1. **Normalize the lore.**
   Convert your notes, Scrivener exports, lore docs, and world text into one structured dataset.
2. **Create three datasets, not one.**
   One for raw canon passages, one for Q&A pairs, and one for style examples.
   Example Q&A pair: "Who are the Fatunik?" → grounded answer from your canon.
3. **Build an evaluator before training.**
   Make 50–100 test questions:
   - factual canon questions
   - ambiguity traps
   - contradiction checks
   - style/voice prompts
4. **Run a baseline local model with retrieval only.**
   Measure how good it already is.
5. **Fine-tune a small adapter.**
   Start with LoRA on a small instruct model, not a base model from scratch. PEFT exists specifically to make that kind of adaptation economical. (Hugging Face)
6. **Import and test the adapter locally.**
   Ollama's docs explicitly cover importing a fine-tuned adapter using a `Modelfile` with `FROM` and `ADAPTER`, which makes local experimentation easier. (Ollama Documentation)

The main traps to avoid are:

- trying to train a model from zero
- throwing messy lore text at the pipeline
- skipping evaluation
- using fine-tuning to compensate for missing retrieval
- trying to pack too much canon into too small a model

For your background, I think this would suit you well because the hard part is less “mystical data science” and more disciplined data engineering:

- corpus design
- provenance
- chunking
- schema
- evaluation
- iteration

That is very much in your wheelhouse.

A concrete first weekend project could be this:

- Export 20–50 lore documents into plain text or markdown
- Build a SQLite or DuckDB catalog with document metadata
- Generate 100 canonical Q&A pairs by hand
- Run a local model against retrieval only
- See where it fails
- Then decide whether to fine-tune

That will tell you very quickly whether the project has legs.

Regarding stage 1, I have quite an extensive lore corpus, written in the form of text and short stories. Will it help to put some of that more into a formal metadata format? As you noted, like sample Q&A or similar? And it sounds like a simple DB would be the place to store it, rather than JSON or YAML?

Stage 2, so a retrieval model would be something kind of like database queries? --> Tell me what is the main town in Baranuk province? that kind of thing? These are the 50-100 test questions?

Stage 3, please explain a little more about PEFT/LoRA, MLX-LM and llama.cpp and Ollama. Trying to wrap my head around all the new terminology and acronyms.

Other: I am thinking starting very small, just an absolute minimum to get some experience, then maybe start over or extend?

Regarding stage 1, I have quite an extensive lore corpus, written in the form of text and short stories. Will it help to put some of that more into a formal metadata format? As you noted, like sample Q&A or similar? And it sounds like a simple DB would be the place to store it, rather than JSON or YAML?

Stage 2, so a retrieval model would be something kind of like database queries? --> Tell me what is the main town in Baranuk province? that kind of thing? These are the 50-100 test questions?

Stage 3, please explain a little more about PEFT/LoRA, MLX-LM and llama.cpp and Ollama. Trying to wrap my head around all the new terminology and acronyms.

Other: I am thinking starting very small, just an absolute minimum to get some experience, then maybe start over or extend?

---

in Spanish ¨matices" (sounds a little like matrices) means ¨nuances¨... I like that. Love your guitar analogy too. Yes. Keeping it simple will be my road to success. This sounds like lots of fun for an old geeky nerd like me. But my main task right now is to finish writing the epic (first) Saskantinon book, __Enclosures__. so my thought on this little side-project is to take a very, umm, Finnish approach. One little step at time....

That’s exactly the right instinct—both technically and creatively.

And your Spanish note is actually perfect:

> *matices* (nuances) ← → matrices

That’s not just a linguistic coincidence—it’s conceptually spot-on. What you’re really doing with this project is teaching a model the **matices** of your world, not just the facts. The facts are easy. The nuance is the whole game.

Your “Finnish approach” is the correct one

For this kind of project:

> slow, deliberate, repeatable beats clever, fast, and ambitious

Especially since your **primary project is the novel**.

The biggest risk here isn’t technical failure—it’s:

> getting sucked into tooling and never finishing *Enclosures*

So the guardrail should be:

- writing comes first
- LLM project stays small, contained, almost *recreational*

A very practical way to structure this

Think of your side-project as:

> **a notebook for thinking about your world**, not a system to build

Define a tiny, fixed scope

For now:

- one region (e.g., Baranuk)
- or one institution (Night Hall, Terpinol, etc.)
- or one theme (the Enclosure itself)

No more than that.

Your “one small step” loop

Here’s a rhythm that won’t derail your writing:

1. Write normally (primary work)
2. Occasionally extract:
    - one lore note
    - one character description
    - one historical fragment
3. Add it to your corpus (clean, structured)
4. Add **one question** about it

That’s it.

No big sessions. No pipelines. No pressure.

After a few weeks, you’ll have:

- 30–50 clean lore entries
- 30–50 test questions

That’s already enough to build Version 0.

Why this works (important)

You avoid two classic traps:

Trap 1: “I need to formalize everything first”

You don’t. You just need **incremental structure**.

Trap 2: “I’ll build the system, then feed it data”

Wrong order.

> The system should grow *out of the data you actually write*

When to actually “build something”

Wait until you have:

- a small but clean corpus
- a handful of good questions

Then:

- build a tiny retrieval prototype
- no training yet
- just see if it can answer correctly

That moment will be:

- motivating
- illuminating
- slightly magical

A creative side benefit (this is the real payoff)

This project will quietly help your writing.

Because it forces you to ask:

- Is this canon or rumor?
- Would two sources disagree here?
- What is actually *known* in this world?

- What would an archivist vs a villager say?

That’s gold for a setting like yours.

A gentle warning (from experience)

If you ever catch yourself thinking:

> “I’ll just spend a few weeks building the perfect system…”

Stop. Immediately.

That’s how side-projects eat novels.

Bottom line

You’ve got it exactly right:

> one small step, tied to real writing, no grand infrastructure

Do that consistently, and in a few months you’ll have:

- a finished book (the real goal)
- and a surprisingly rich dataset ready for experimentation

When you’re ready, we can take that small corpus and turn it into a **first working “Saskan Lands assistant”**—no overengineering, just something that answers your questions about your own world.

I expect I will use ChatGPT + Claude to help extract from my large, verbose core lore stories to create / model / format metadata into a more useful form.

Yes—this is exactly where ChatGPT and Claude shine. Think of them as **extract-transform assistants** for your lore, not as sources of truth.

The key is to **constrain the task tightly** so they *extract* from your text instead of inventing.

A simple, reliable workflow

1) Keep your source sacred

- Your story/lore text = ground truth
- Don’t edit it to “fit the model”
- Always keep a link back to the original passage

---

2) Use a fixed extraction schema

Give the model a **very explicit template** and force it to stick to it.

For example:

```
{
  "title": "",
  "summary": "",
  "region": [],
  "places": [],
  "characters": [],
  "factions": [],
  "era": "",
  "canon_level": "",
  "truth_status": "",
  "key_events": [],
  "claims": [
    {
      "statement": "",
      "confidence": "",
      "source_span": ""
    }
  ]
}
```

---

3) Use a strict prompt (important)

Something like:

> Extract structured metadata from the text below.
> Only include information explicitly stated or strongly implied.
> Do not invent details.
> If uncertain, leave the field empty or mark as “unknown”.
> Include short source spans (quotes) for each claim.

That last line—**source spans**—is the difference between useful and dangerous output.

What the models are best at here

ChatGPT

- precise formatting
- consistent JSON output
- good at following strict rules

Claude

- better at long passages
- better at subtle interpretation
- good at narrative summaries

Use both if you like:

- Claude → first pass extraction
- ChatGPT → normalization / formatting

A concrete “small step” example

Take one lore passage.

Ask:

> “Give me 3–5 factual claims from this passage with quotes.”

Then:

> “Convert this into structured metadata using the schema.”

Then:

- skim it yourself (fast sanity check)
- store it

That’s a 2–3 minute loop.

What to avoid (this matters)

❌ “Summarize this world”

Too vague → hallucinations

❌ “Fill in missing fields”

You’ll get invented canon

❌ **giant multi-document prompts**

You lose traceability

A trick you'll probably appreciate

Add a field like:

`"extraction_confidence": 0.0–1.0`

Not because it's "scientific," but because it forces the model to signal uncertainty—and you to notice it.

Another useful pattern (very powerful later)

Have the model extract **conflicts**:

> "List any statements in this passage that could contradict other sources."

That feeds directly into:

- richer lore
- better evaluation questions
- more interesting model behavior later

Where this leads

After a few weeks of light use:

- you'll have clean, structured lore
- with traceable origins
- and minimal hallucination risk

At that point:

- building retrieval becomes easy
- fine-tuning becomes meaningful

Bottom line

You're using the tools exactly right:

> LLMs = assistants for structuring your knowledge
> You = authority on truth

Keep that boundary, and this will stay productive instead of turning into a mess.

And yes—this is *exactly* the kind of quietly nerdy, deeply satisfying side project that pairs well with writing a big world.

I have found that working with both ChatGPT and Claude and making sure I am doing my human part too (good prompts, thorough review, correcting errors, taking main responsibility for content, insisting on test cases, etc.) ends up making for a better result.

Exactly. That's the mature way to use these tools, and it maps very cleanly to how this kind of project succeeds.

What you're really doing is shifting from:

> "ask a model for answers"

to

> **build a small, testable system where models are components**

That's a very different mindset—and it's why your results feel better.

The pattern you've already landed on (and why it works)

You've got all the right pieces in place:

- **good prompts** → constrain behavior
- **human review** → enforce truth
- **corrections** → improve data quality
- **test cases** → measure progress

That's basically:

> data engineering + QA + iterative modeling

Which, frankly, is more important than the model choice.

One small upgrade (high leverage)

Since you mentioned test cases, I'd suggest one tweak that pays off quickly:

Treat your eval set like code

Even if it’s just a CSV or SQLite table:

- don’t overwrite answers casually
- track changes
- add notes when something fails

Example fields:

- `question`
- `expected_answer`
- `model_answer`
- `pass_fail`
- `failure_type` (wrong fact, hallucination, incomplete, style, etc.)

This gives you something like:

> a unit test suite for your world

And once you have that:

- you can compare retrieval vs fine-tuned
- you can see regressions
- you can actually *measure improvement*

Another subtle win you’re already getting

By forcing yourself to:

- extract
- structure
- validate

…you’re tightening your own worldbuilding.

You’ll start noticing things like:

- “this faction’s timeline doesn’t quite line up”
- “this term is used inconsistently”
- “this ‘fact’ is actually just one character’s belief”

That’s exactly the kind of depth your setting thrives on.

Where this leads (when you’re ready)

Down the line, your system becomes:

- corpus (clean, structured, versioned)
- retrieval (ground truth access)
- evaluation (test set)
- optional fine-tuning (style + behavior)

At that point, the model is almost the least interesting part.

And importantly…

You’re keeping the right priority:

> finish *Enclosures* first

This side project should:

- support your thinking
- not compete with your writing

If it ever starts feeling like the main event, that’s your cue to scale it back.

Bottom line

You’re not just “using AI tools.”

You’re:

> building a disciplined, human-in-the-loop system

That’s why it’s working—and why it will keep working as things evolve.

When you’re ready, we can wire up a **tiny first-pass retrieval prototype**—something you can run locally without it turning into a weekend-eating monster.

Longer term it would be nice to have an AI assistant / tool that is deeply connected to the Saskan lore, easier to look things up for continuity editing, etc. and perhaps a way to build up some neural networks / graphs ?? that might help to discover/explore plot lines, etc.

Yes—that’s a very good long-term target, and it naturally splits into **two different tools**, not one.

1. The continuity assistant

This is the practical one. It answers things like:

- Who was in Baranuk at this point in the timeline?
- Did I already say whether X knew Y?
- What name did I use for that river in chapter 4?
- Is this institution older than that one?
- What would a Night Hall scholar believe about this event?

This is mostly a **retrieval + metadata + timeline** problem, not a grand AI problem. And that is good news, because it means it is tractable.

The backbone would be:

- your source texts
- structured metadata
- chunked passages
- timelines
- entity tables for people, places, factions, institutions
- relationship links
- search and retrieval on top

That alone would already be extremely useful for continuity editing.

2. The exploratory assistant

This is the more speculative one. It helps with things like:

- Which factions are underused in the current draft?
- What characters have indirect but interesting connections?
- Where are there pressure points in the political or ecological system?
- Which unresolved tensions could plausibly become plot lines?
- What themes or motifs recur in this region but not that one?

That is where graph structures, embeddings, and eventually neural-ish tools become interesting.

But I would keep this grounded. A lot of “discover plot lines” can come from a well-built **knowledge graph** plus some lightweight analysis, long before you need anything exotic.

For example, if you model:

- characters
- places
- factions
- events
- beliefs
- conflicts
- dependencies

and link them, then you can already ask very rich questions:

- Show all events touching both the Dominion and Mobalbesq
- Which characters connect the Night Hall to Outer Lands trade networks?
- What events in the famine era have unresolved causal links?
- Which places are mentioned often but never shown directly in scene?

That is already powerful.

My blunt recommendation

Build in this order:

Phase A — lookup tool

A clean lore catalog for continuity and canon lookup.

Phase B — relationships

Add links between entities, events, and documents.

Phase C — exploratory queries

Ask “what is connected to what?” and “what is missing?”

Phase D — model-assisted ideation

Only then let an LLM help suggest plausible plot tensions, hidden links, thematic recurrences, and so on.

That way the AI is exploring **your structure**, not free-associating nonsense.

Knowledge graph versus neural network

These are different beasts.

Knowledge graph

Explicit, inspectable, editable.

Example:

- Stan → member_of → Agency
- Enclosure → caused → pollination crisis
- Night Hall → located_in → Byenung

This is excellent for continuity and analysis.

Neural network / embeddings

Pattern-based, fuzzy, associative.

Good for:

- semantic similarity
- clustering themes
- finding related passages
- surfacing latent connections

Not good for:

- canonical truth by itself
- transparency

So for your use case, I would strongly favor:

> **graph for truth and structure**
> **embeddings/LLMs for search, clustering, and suggestion**

That is a very sane division of labor.

What this could become later

Something like a local “Saskan console”:

- search bar
- timeline view
- entity cards
- graph of relationships
- continuity checker
- lore Q&A grounded in canon
- “show contradictions / ambiguities / missing links”
- optional creative mode for brainstorming

That would be genuinely useful.

Tiny first step that points in that direction

When you extract metadata, do not just store flat tags. Also store links like:

- character appears_in document
- character affiliated_with faction
- place located_in region
- event involves faction
- event occurs_in era
- claim sourced_from document chunk

That way, even before you have a formal graph database, your SQLite data can already behave graph-like.

Important caution

Do not let “plot discovery” turn into “outsourcing imagination.”

The sweet spot is:

- the system surfaces tensions, patterns, absences, odd connections
- you decide what matters dramatically

That keeps the work yours.

Bottom line

You’re describing a very good long-range architecture:

- **continuity assistant** for canon truth
- **graph/relationship layer** for structure
- **LLM layer** for retrieval, summarization, and exploration

That is not pie-in-the-sky. It is quite buildable if you take it one layer at a time.

Next good step would be to sketch the **minimum data model** for:

- entities
- documents
- chunks
- events
- relationships
- test questions

and keep it small enough that it supports *Enclosures* rather than becoming a second novel.

I can see where asking the AI tools to pull some metadata from the epic texts will also be useful. Maybe if I get bored even write a bit of my own text analysis in Python?

That's exactly the right instinct—and honestly, the Python side is where this becomes *your* system instead of just "using AI tools."

You don't need anything fancy to start. A few small, purposeful scripts will give you a lot of leverage.

Where your own Python fits (high value, low effort)

Think of Python as doing the **boring, reliable work**:

- cleaning text
- chunking documents
- tracking provenance
- running evaluations
- spotting patterns

Let the LLMs do:

- extraction
- summarization
- interpretation

That division keeps things sane.

---

3 small scripts worth writing early

1) Chunker (foundation piece)

Take a document → split into usable chunks.

Simple version:

```
def chunk_text(text, max_len=800):
    words = text.split()
    chunks = []
    current = []

    for w in words:
        current.append(w)
        if len(current) >= max_len:
```

```
            chunks.append(" ".join(current))
            current = []

    if current:
        chunks.append(" ".join(current))

    return chunks
```

Later you'll improve this (paragraph-aware, overlap, etc.), but this is enough to start.

---

2) Metadata loader

Load your extracted JSON into a DB (SQLite/DuckDB).

```
import sqlite3
import json

conn = sqlite3.connect("lore.db")

def insert_doc(doc):
    conn.execute("""
        INSERT INTO documents (doc_id, title, region, era, text)
        VALUES (?, ?, ?, ?, ?)
    """, (
        doc["doc_id"],
        doc["title"],
        doc.get("region"),
        doc.get("era"),
        doc["text"]
    ))
```

Nothing fancy. Just make it repeatable.

---

3) Evaluation runner (this is gold)

Given:

- a question
- retrieved chunks
- model answer

Store results:

```
def record_result(q, expected, actual, passed, notes=""):
    conn.execute("""
        INSERT INTO eval_results
        (question, expected, actual, passed, notes)
        VALUES (?, ?, ?, ?, ?)
    """, (q, expected, actual, passed, notes))
```

This is how you avoid "it feels better" vs "it is better."

One fun analysis you can do early

This is very “you” given your background.

Entity frequency + co-occurrence

- how often does each character appear?
- which characters co-occur?
- which places are underused?

Even a crude pass gives insight.

Example idea:

```
from collections import Counter

character_counts = Counter()

for chunk in chunks:
    for name in character_list:
        if name in chunk:
            character_counts[name] += 1
```

Later:

- build co-occurrence matrix
- visualize relationships
- identify clusters

That’s already proto-“graph thinking.”

Another useful one: terminology consistency

Check things like:

- “Night Hall” vs “NightHall”
- “Fatune” vs “Fatunik”
- alternate spellings

Simple regex scans can catch a lot.

Where this connects back to your AI workflow

Once you have even basic Python tooling:

- LLM extracts → JSON
- Python validates → flags issues
- You review → correct

That loop is powerful.

Keep it small (seriously)

You only need:

- a few scripts
- a tiny DB
- some structured text

Resist the urge to build:

- full pipelines
- frameworks
- dashboards

Those come later, if ever.

A good “weekend-sized” milestone

- take 5–10 lore documents
- chunk them
- extract metadata with ChatGPT/Claude
- store in SQLite
- run 10 test questions manually

If that works:

> you already have Version 0 of your system

Bottom line

Yes—write some Python.

Not to be clever, but to:

> make your data reliable, inspectable, and testable

That’s where your background gives you a real edge over most people doing this kind of project.

And honestly, this is the fun part:
small scripts, clean data, incremental insight… no hype required.

There is really little need to go beyond SQLite, right? I have plenty of good code models for using SQLAlchemy to work with SQLite, so if I ever decide to port to Postgres or whatever, not a big deal.

Short answer: **you're absolutely fine with SQLite**—and for this project, it's the *right* choice.

If you were designing this from scratch as a "serious system," you might be tempted to overbuild. Don't.

Why SQLite is exactly right here

For your use case:

- single user
- local workflows
- modest dataset (even "large lore corpus" is tiny by DB standards)
- iterative, exploratory work

SQLite gives you:

- zero setup
- fast enough performance
- full SQL power
- easy backups (it's just a file)
- portability

That last one matters more than people realize:

> you can copy your entire "world brain" with one file

When SQLite starts to creak (so you know the boundary)

You'd only need to move beyond it if you hit something like:

- multiple concurrent users writing

- very large-scale embeddings (millions of vectors)
- heavy parallel workloads
- need for a dedicated graph database

You are nowhere near that.

SQLAlchemy: good call

Using SQLAlchemy buys you:

- portability (SQLite → Postgres later if needed)
- cleaner schema evolution
- less raw SQL boilerplate

But I'd keep it **lightweight**:

- don't build an elaborate ORM layer
- treat it as a thin abstraction

One optional upgrade to consider

If you want slightly more analytical power:

DuckDB

It's similar to SQLite but better for:

- analytics
- columnar queries
- working with larger datasets

That said:

> SQLite is simpler and perfectly sufficient

I wouldn't switch unless you feel a specific limitation.

What about vector search?

This is the only area where people think they need something else.

Reality:

You can stay in SQLite and still do this:

- store embeddings as blobs or arrays
- use extensions like:
    - `sqlite-vss`
    - or simple cosine similarity in Python

For your scale:

> Python + SQLite is more than enough

No need for Pinecone/Weaviate/etc.

What about graph databases?

Tempting, but premature.

You can model relationships in SQLite just fine:

- `entities`
- `relationships` table
- joins

Example:

```
relationships(
  source_id,
  target_id,
  relationship_type
)
```

That gets you 80–90% of graph usefulness.

Only consider Neo4j/etc. if:

- you *really* need complex traversal queries
- and you feel pain doing them in SQL

A practical architecture (clean and durable)

- **Markdown/text files** → source of truth
- **SQLite DB** → indexed, structured view
- **Python scripts** → ingestion, validation, analysis
- **LLMs** → extraction + interpretation

That’s it.

No services. No infra. No cloud.

The real advantage of staying simple

You get:

- fast iteration
- no operational overhead
- easy refactoring
- low cognitive load

Which matters because:

> your real project is the book

Bottom line

You're making the right call:

> SQLite + SQLAlchemy = durable, flexible, zero-friction foundation

You can always scale up later—but odds are, you won't need to.

If you want next, I can sketch a **minimal SQLAlchemy schema** that matches everything we discussed (documents, chunks, entities, relationships, eval set) without turning into a monster.

I have looked at DuckDB a few times but was leery about adding new tools if I did not really need them. Having said that, I do get the advantages of columnar queries for analytics. I have worked with every large scale DB under the Sun at one time or another, even (shudder....) Teradata. So I imagine I could handle it OK. LOL.

Your instinct is right: **don't add DuckDB yet**. You don't *need* it to get real progress.

Given your background, you already know the pattern:

> the tool should follow the workload, not the other way around

Right now your workload is:

- small-to-medium corpus
- iterative schema changes
- lots of reads, occasional writes
- some light analysis

That is **SQLite territory all day long**.

Where DuckDB *actually* earns its keep (so you know when to switch)

DuckDB shines when you start doing things like:

- wide scans over lots of rows (“show me all chunks with X across everything”)
- aggregations over many dimensions (era × faction × region × time)
- ad-hoc analytical queries
- joining lots of semi-structured data
- working with Parquet/Arrow datasets

In your project, that corresponds to a later phase like:

> “What patterns exist across the entire Saskan corpus?”

Examples:

- which factions co-occur most often across eras
- which regions are underrepresented in narrative coverage
- clustering themes across documents
- frequency shifts over timeline

That’s where columnar execution helps.

What I would do (pragmatic path)

Phase 1 (now)

- SQLite only
- SQLAlchemy if you like
- focus on:
    - ingestion
    - chunking

- retrieval
- evaluation

No DuckDB.

Phase 2 (when curiosity kicks in)

When you *feel* like doing analysis, don't migrate—just **add DuckDB alongside**.

Example pattern:

- SQLite → operational store
- DuckDB → read-only analytical layer

You can literally:

```
import duckdb
con = duckdb.connect()

con.execute("""
    SELECT region, COUNT(*)
    FROM sqlite_scan('lore.db', 'chunks')
    GROUP BY region
""")
```

DuckDB can query SQLite directly. No migration needed.

Why this hybrid approach works

- SQLite stays your **system of record**
- DuckDB becomes your **lab bench**

You avoid:

- schema churn headaches
- premature optimization
- rewriting code

And you get:

- fast analytics when you want it

About your Teradata flashback 😄

What you're building here is basically:

> a tiny, humane, single-user data warehouse for a fictional world

So yes, your instincts from that world apply—but scaled down:

- clean schema
- controlled ingestion
- testable queries
- no overengineering

And thankfully:

- no query planners from hell
- no spool space errors
- no midnight pager duty

A simple rule of thumb

Stay with SQLite until you hit one of these thoughts:

- “This query feels slow because I’m scanning everything”
- “I want to slice this 12 different ways quickly”
- “I’m exporting to pandas just to do analysis”

At that point:

> add DuckDB, don’t replace SQLite

Bottom line

You’re thinking about this exactly right:

- resist new tools unless needed
- understand where they fit
- add them surgically

For now:

> SQLite = foundation
> DuckDB = optional future microscope

If you want later, I can show a **very small example of using DuckDB to analyze your lore (co-occurrence, timeline density, etc.)** without changing your core system.